

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Лабораторная работа №5
«Реализация детерминированного левого анализатора для
грамматики $LL(1)$ »
по дисциплине
«Математическая логика и теория формальных языков»
Вариант 11

Студент,

группы 5130201/30101

_____ Мелешенко С.И.

Преподаватель

_____ Востров А. В.

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Математическое описание	4
1.1 Исходная грамматика	4
1.2 Порождающие грамматики	4
1.3 Классификация грамматик по Хомскому	5
1.4 LL(k)-грамматики	6
1.5 LL(1)-грамматики	6
1.6 s-грамматики	7
1.7 Множества FIRST и FOLLOW для заданной грамматики	7
1.8 Таблица выбора	8
1.9 LR(k)-грамматики	8
1.10 LR(1)-грамматики	8
1.11 Матрица отношений предшествования	9
1.12 Алгоритм разбора предложений	11
1.13 Алгоритм генерации цепочек	11
1.14 Семантические действия	12
1.15 Пример вывода цепочки	12
2 Особенности реализации	13
2.1 Класс GrammarAnalyzer	13
2.1.1 Метод GrammarAnalyzer (конструктор)	13
2.1.2 Метод computeFirstSets()	14
2.1.3 Метод computeFollowSets()	15
2.1.4 Метод buildParsingTable()	16
2.1.5 Метод parse(String input)	17
2.1.6 Метод generate(Random rand)	19
2.2 Класс Main	19
2.2.1 Метод main(String[] args)	19
2.3 Интерфейс SemanticActions	20
3 Результаты работы	22
Заключение	26
Список литературы	27

Введение

Построить множества FIRST и FOLLOW для каждого нетерминала грамматики и таблицу выбора (lookup table) и матрицу отношений предшествования. На их основе реализовать детерминированный левый анализатор для проверки принадлежности цепочки грамматике LL(1) (или правый анализатор LR(1), если это эффективнее). Реализовать генерацию цепочки. Назначить произвольные семантические действия для заданных продукций (например, генерацию машинноориентированного кода, движение робота или алгоритма выхода из лабиринта). Весь лог распознавания выводится на экран.

Вариант 11:

$$\begin{aligned} S &\rightarrow aB \\ B &\rightarrow aBB \mid b \end{aligned}$$

Задачи:

1. Построить множества FIRST и FOLLOW для каждого нетерминала грамматики.
2. Построить таблицу выбора (lookup table) и матрицу отношений предшествования.
3. На основе построенных множеств и таблиц реализовать детерминированный левый анализатор для проверки принадлежности цепочки грамматике LL(1).
4. Реализовать генерацию цепочек, принадлежащих языку.
5. Назначить произвольные семантические действия для заданных продукций (в работе реализована генерация машинно-ориентированного кода для стековой машины: инструкции PUSH 1, DUP, PUSH 2).
6. Обеспечить вывод подробного лога процесса распознавания (состояние стека, текущий символ, применяемое правило и генерируемая инструкция).

1 Математическое описание

1.1 Исходная грамматика

Задана следующая контекстно-свободная грамматика:

$$S \rightarrow aB, \quad B \rightarrow aBB \mid b.$$

Терминальный алфавит: $T = \{a, b\}$.

Нетерминальный алфавит: $N = \{S, B\}$.

Начальный символ: S .

Множество продукций: $P = \{S \rightarrow aB, B \rightarrow aBB, B \rightarrow b\}$.

Грамматика не содержит ε -правил, не имеет левой рекурсии (в правиле $B \rightarrow aBB$ первый символ — терминал a), поэтому она уже пригодна для построения LL(1)-анализатора без предварительных преобразований.

Грамматика также является LR(1), поскольку:

- Грамматика LL(1) (по критерию непересекающихся FIRST).
- Поскольку класс LL(1) является подклассом LR(1), грамматика автоматически является LR(1).

Таким образом, грамматика принадлежит одновременно к классам LL(1) и LR(1).

Кроме того является s-грамматикой, т.к. альтернативы одного и того же нетерминала начинаются различными терминалами, выбор однозначен.

Почему выбран LL(1), а не LR(1)? Для данной грамматики LL(1) обладает следующими преимуществами:

- Таблица LL(1) имеет размер $|N| \times (|T| + 1)$, тогда как таблица LR(1) была бы больше из-за множества состояний.
- Стек хранит символы грамматики, управляющая таблица строится напрямую по множествам FIRST и FOLLOW.
- Каждому правилу грамматики можно сопоставить действие, которое выполняется в момент его применения — подходит для генерации кода или интерпретации.

Для решения поставленной задачи LL(1) является более удобным и наглядным выбором, реализация s-анализатора не предумсотрена заданием. Далее LL(1) и LR(1) будут рассмотрены подробнее.

1.2 Порождающие грамматики

Определение. Порождающая грамматика Хомского — это четвёрка $G = (N, T, P, S)$, где:

- N — конечный алфавит нетерминальных символов;
- T — конечный алфавит терминальных символов;
- P — конечное множество продукций (правил) вида $\alpha \rightarrow \beta$, где $\alpha \in (N \cup T)^+$ и $\beta \in (N \cup T)^*$;
- $S \in N$ — начальный символ.

Язык, порождаемый грамматикой G :

$$L(G) = \{w \in T^* \mid S \Rightarrow^* w\}.$$

где:

- T^* — множество всех конечных последовательностей (включая пустую) символов из терминального алфавита T ;
- w — произвольный элемент этого множества, то есть любая строка, составленная только из терминальных символов;
- $\Rightarrow^*$ обозначает вывод за ноль или более шагов.

1.3 Классификация грамматик по Хомскому

В зависимости от вида правил грамматики делятся на четыре типа (табл. 1).

Таблица 1. Иерархия Хомского

Тип	Вид правил	Распознающее устройство
0	$\alpha \rightarrow \beta$ (неограниченные)	Машина Тьюринга
1	$\gamma A \delta \rightarrow \gamma \beta \delta$ (контекстно-зависимые)	Линейно-ограниченный автомат
2	$A \rightarrow \beta$ (контекстно-свободные)	Автомат с магазинной памятью
3	$A \rightarrow aB, A \rightarrow a, A \rightarrow \varepsilon$ (регулярные)	Конечный автомат

Здесь использованы следующие обозначения:

- A, B — произвольные нетерминалы ($A, B \in N$);
- a — произвольный терминальный символ ($a \in T$);
- $\alpha, \beta, \gamma, \delta$ — произвольные цепочки из терминалов и нетерминалов ($\alpha, \beta, \gamma, \delta \in (N \cup T)^*$), причём в правилах типа 1 $\gamma A \delta \rightarrow \gamma \beta \delta$ цепочка β непустая ($\beta \in (N \cup T)^+$);
- ε — пустая цепочка.

Грамматика варианта 11 относится к типу 2 (контекстно-свободным), так как левая часть каждого правила состоит из одного нетерминала. При этом она не является регулярной (тип 3), поскольку правило $B \rightarrow aBV$ содержит два нетерминала в правой части.

1.4 LL(k)-грамматики

Расшифровка обозначения. Буквы в названии LL(k) означают:

- первая **L** (left) — входная цепочка читается слева направо;
- вторая **L** (leftmost) — строится левый (левосторонний) вывод входной цепочки;
- **k** — количество символов предпросмотра (lookahead), используемых на каждом шаге для принятия решения.

Формальное определение. Грамматика G имеет свойство $LL(k)$, если для любых двух левых выводов

$$S \Rightarrow^* wAx \Rightarrow w\alpha x \Rightarrow^* wu, \quad S \Rightarrow^* wAx \Rightarrow w\beta x \Rightarrow^* wv,$$

где $w \in (V_T \cup V_N)^*$, $A \in V_N$, $\alpha, \beta, x \in (V_T \cup V_N)^*$, $u, v \in V_T^*$, выполняется условие:

$$\text{FIRST}_k(u) = \text{FIRST}_k(v) \implies \alpha = \beta.$$

- S — начальный символ грамматики;
- V_T — множество терминальных символов;
- V_N — множество нетерминальных символов;
- $w, x, \alpha, \beta \in (V_T \cup V_N)^*$ — произвольные цепочки из терминалов и нетерминалов (сентенциальные формы);
- $A \in V_N$ — нетерминал, который заменяется на шаге вывода;
- $u, v \in V_T^*$ — терминальные цепочки (слова языка);
- $\Rightarrow^*$ — вывод за ноль или более шагов;
- $\text{FIRST}_k(u)$ — множество первых k символов цепочки u (если $|u| < k$, то вся цепочка).

1.5 LL(1)-грамматики

Для практического применения наиболее важен случай $k = 1$. Принятие решения о выборе правила для нетерминала A основывается на функциях FIRST и FOLLOW.

$\text{FIRST}(\alpha)$ — множество терминалов, с которых могут начинаться цепочки, выводимые из α .

$\text{FOLLOW}(A)$ — множество терминалов, которые могут следовать за A в какой-либо сентенциальной форме.

Грамматика является $LL(1)$, если для каждого нетерминала A с продукциями $A \rightarrow \alpha_1 \mid \alpha_2 \mid \dots \mid \alpha_n$ выполняется условие:

$$\text{FIRST}(\alpha_i) \cap \text{FIRST}(\alpha_j) = \emptyset \text{ для всех } i \neq j;$$

То есть множества выбора для всех альтернатив каждого нетерминала не пересекаются.

Грамматика варианта 11 является LL(1)-грамматикой.

1.6 s-грамматики

Определение. s-грамматики (от англ. *simple* — простой) являются самым простым обобщением автоматных грамматик. Правила s-грамматики имеют вид

$$A \rightarrow \alpha\beta,$$

где α — терминал (ключ правила), β — произвольная цепочка из терминалов и нетерминалов. При этом у всех альтернатив одного и того же нетерминала ключи различны.

Для любого неавтоматного языка, если альтернативы одного и того же нетерминала начинаются различными терминалами, то выбор правил подстановки однозначен. Такие грамматики и называют s-грамматиками.

Это позволяет построить простой детерминированный нисходящий алгоритм распознавания для s-грамматик.

1.7 Множества FIRST и FOLLOW для заданной грамматики

Вычислим множества FIRST для всех символов грамматики.

Для терминалов: $\text{FIRST}(a) = \{a\}$, $\text{FIRST}(b) = \{b\}$.

Для нетерминалов:

$$\begin{aligned}\text{FIRST}(B) &= \text{FIRST}(aBB) \cup \text{FIRST}(b) = \{a\} \cup \{b\} = \{a, b\}, \\ \text{FIRST}(S) &= \text{FIRST}(aB) = \{a\}.\end{aligned}$$

Множества FOLLOW:

$$\text{FOLLOW}(S) = \{\$ \};$$

Из правила $S \rightarrow aB$: $\text{FOLLOW}(B) \supseteq \text{FOLLOW}(S) = \{\$ \}$;

Из правила $B \rightarrow aBB$: первый B за ним следует $B \Rightarrow \text{FOLLOW}(B) \supseteq \text{FIRST}(B) = \{a, b\}$.

Так как ϵ не выводится, других добавлений нет. Итоговые множества:

$$\text{FOLLOW}(B) = \{a, b, \$ \}, \quad \text{FOLLOW}(S) = \{\$ \}.$$

Результаты сведены в таблицу 2.

Таблица 2. Множества FIRST и FOLLOW для грамматики $S \rightarrow aB$, $B \rightarrow aBB \mid b$

Нетерминал	FIRST	FOLLOW
S	$\{a\}$	$\{\$ \}$
B	$\{a, b\}$	$\{a, b, \$ \}$

1.8 Таблица выбора

Заполняем таблицу $T[A, t]$ по описанному алгоритму:

- Для $S \rightarrow aB$: $\text{FIRST}(aB) = \{a\} \rightarrow T[S, a] = S \rightarrow aB$.
- Для $B \rightarrow aBB$: $\text{FIRST}(aBB) = \{a\} \rightarrow T[B, a] = B \rightarrow aBB$.
- Для $B \rightarrow b$: $\text{FIRST}(b) = \{b\} \rightarrow T[B, b] = B \rightarrow b$.

Все остальные ячейки пусты. Таблица представлена в таблице 3.

Таблица 3. Таблица выбора

	a	b	$\$$
S	$S \rightarrow aB$	—	—
B	$B \rightarrow aBB$	$B \rightarrow b$	—

Каждая ячейка содержит не более одного правила, следовательно, грамматика является LL(1).

1.9 LR(k)-грамматики

LR(k)-грамматики — наиболее широкий класс контекстно-свободных грамматик, допускающих эффективный восходящий детерминированный синтаксический анализ.

Расшифровка обозначения:

- **L** (left) — анализируемая цепочка (сентенциальная форма) просматривается слева направо;
- **R** (right) — восстанавливается правый канонический вывод цепочки языка;
- **k** — количество символов, которые алгоритм свертки просматривает вперёд (lookahead) для обнаружения очередной связки.

LR(k)-грамматики были введены Д. Кнудом и являются сейчас основным классом грамматик, для которого строятся практические производственные компиляторы.

1.10 LR(1)-грамматики

Чтобы грамматика была LR(1), необходимо распознавать вхождение правой части правила вывода, просмотрев всё, что выведено из этой правой части, и текущий символ входной цепочки. Это требование существенно менее строгое, чем требование для LL(1)-грамматики, когда необходимо определить применимое правило, видя только первый символ, выводимый из его правой части.

LR(1)-анализ обладает следующими свойствами:

- это наиболее мощный метод анализа без возвратов;
- может быть реализован довольно эффективно;
- LR(1)-анализаторы могут быть построены для практически всех конструкций языков программирования;
- класс грамматик, анализируемых LR(1)-методом, строго включает класс грамматик, анализируемых нисходящими предсказывающими анализаторами типа LL(1).

Однако автомат LR(1) разбора, хотя и строится по ясному и понятному алгоритму, является чрезвычайно громоздким: даже при $k = 1$ ситуаций, отличающихся своими контекстами, очень много, а их подмножеств — огромное число даже для простых грамматик. Поэтому на практике часто используют упрощённые варианты, такие как SLR(1) или LALR(1).

1.11 Матрица отношений предшествования

Для построения матрицы отношений предшествования грамматику дополняют новым начальным нетерминалом S' и правилом $S' \rightarrow \#S\#$, где $\#$ — специальный маркер границ (не входит в алфавит T). Полученная грамматика имеет вид:

$$S' \rightarrow \#S\#, \quad S \rightarrow aB, \quad B \rightarrow aBB \mid b.$$

Бинарные отношения $\{< \cdot, \doteq, \cdot >\}$ для пары символов X, Y (терминалов или нетерминалов) вводятся следующим образом:

- $X \doteq Y$ (одновременная сворачиваемость), если существует правая часть какой-либо продукции грамматики, в которой эти два символа стоят рядом.
- $X < \cdot Y$ (предшествование), если существует правило $A \rightarrow \alpha X M \beta$ и $M \Rightarrow^+ Y \gamma$ (т.е. Y — первый символ цепочки, выводимой из M).
- $X \cdot > Y$ (следование), если существует правило $A \rightarrow \alpha M Y \beta$, $M \Rightarrow^+ \gamma X$ (т.е. X — последний символ цепочки, выводимой из M), причём Y — терминал.

Вычислим эти отношения для символов $\{\#, S, B, a, b\}$.

Отношение $\doteq$:

- из $S' \rightarrow \#S\#$: $\# \doteq S$, $S \doteq \#$;
- из $S \rightarrow aB$: $a \doteq B$;
- из $B \rightarrow aBB$: $a \doteq B$, $B \doteq B$;
- из $B \rightarrow b$: нет пар.

Отношение $< \cdot$:

- из $S' \rightarrow \#S\#$: $\#$ стоит перед S ; S может начинаться с a ($\text{FIRST}(S) = \{a\}$), поэтому $\# < \cdot a$;
- из $S \rightarrow aB$: a стоит перед B ; B начинается с a или b , следовательно $a < \cdot a$, $a < \cdot b$;
- из $B \rightarrow aBB$: первый a перед B — те же $a < \cdot a$, $a < \cdot b$; также B перед B : B может начинаться с a или b , поэтому $B < \cdot a$, $B < \cdot b$.

Отношение $\cdot >$:

- из $S' \rightarrow \#S\#$: S стоит перед $\#$; цепочки, выводимые из S , оканчиваются на b (например, ab , $aabb$, $aaabbb$), поэтому $b \cdot > \#$;
- из $S \rightarrow aB$: после B нет символов, отношения нет;
- из $B \rightarrow aBB$: второй B стоит в конце правила, поэтому непосредственного отношения нет, но B может оканчиваться на b , что уже учтено в $b \cdot > \#$.

Построим матрицу отношений предшествования (табл. 4). Пустые ячейки означают отсутствие отношения.

Таблица 4. Матрица отношений предшествования

	$\#$	S	B	a	b
$\#$				$< \cdot$	
S	$\cdot >$				
B				$< \cdot$	$< \cdot$
a			$\doteq$	$< \cdot$	$< \cdot$
b	$\cdot >$				

Согласно определению, КС-грамматика называется грамматикой простого предшествования, если:

- она не содержит ε -продукций;
- она не содержит различных продукций с одинаковыми правыми частями;
- матрица отношений предшествования однозначна (не содержит различных отношений для каждой пары символов).

Грамматика варианта 11 является грамматикой простого предшествования.

1.12 Алгоритм разбора предложений

Перед запуском алгоритма разбора в стек помещаются два символа:

- специальный символ $\$$ — признак конца входной цепочки;
- начальный символ грамматики (на вершину стека).

Синтаксический анализатор выполняет три различных вида действий в зависимости от того, находится ли на вершине стека нетерминал, терминал или специальный символ $\$$.

1. Если на вершине стека — нетерминал, то в таблице синтаксического анализа ищется правило грамматики, находящееся на пересечении строки и столбца, соответствующих этому нетерминалу на вершине стека и текущему входному символу. Если правило найдено, то нетерминал удаляется из стека, а правая часть правила помещается в стек в обратном порядке (чтобы левый символ оказался на вершине). Если ячейка пуста — ошибка.
2. Если на вершине стека — терминал, то синтаксический анализатор сравнивает его с текущим входным символом. Если они равны, то входной символ считывается, а соответствующий символ из вершины стека удаляется. Иначе — ошибка.
3. Если на вершине стека — специальный символ $\$$ и текущий символ на ленте также равен $\$$, то синтаксический анализатор сообщает об успешном распознавании цепочки. В противном случае — ошибка.

Эти шаги повторяются до тех пор, пока не произойдёт останов. Алгоритм работает за линейное время $O(n)$, где n — длина входной строки.

1.13 Алгоритм генерации цепочек

Генерация цепочек выполняется рекурсивно:

1. Начинаем с начального символа S .
2. Для текущего нетерминала случайным образом выбираем одно из его правил.
3. Рекурсивно раскрываем каждый символ правой части.
4. Процесс завершается, когда в цепочке не остаётся нетерминалов.

Так как грамматика не содержит ε -правил, рекурсия всегда заканчивается, и сгенерированная строка гарантированно принадлежит языку.

1.14 Семантические действия

Каждой продукции можно сопоставить семантическое действие, выполняемое в момент применения правила при разборе. В данной лабораторной работе в качестве семантических действий выбрана генерация кода для стековой машины (простой ассемблер). Применяются следующие инструкции:

- $S \rightarrow aB$: PUSH 1 — положить константу 1 на стек.
- $B \rightarrow aBB$: DUP — дублировать вершину стека.
- $B \rightarrow b$: PUSH 2 — положить константу 2 на стек.

Пример: для строки **aabb** генерируется программа:

PUSH 1, DUP, PUSH 2, PUSH 2,

которая при выполнении на стековой машине вычисляет некоторое значение (например, $1 \cdot 2 \cdot 2 = 4$).

1.15 Пример вывода цепочки

Построим левый вывод терминальной цепочки $aaabbb$ (Рис. 1).

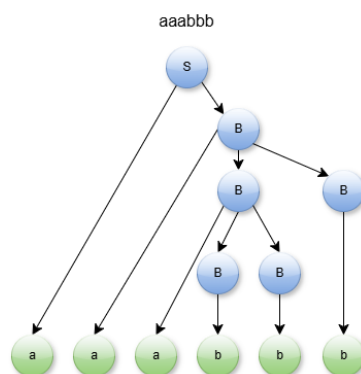


Рис. 1. Дерево разбора строки aaabbbb

$$\begin{aligned}
S &\Rightarrow aB \\
&\Rightarrow a(aBB) && (\text{применяем } B \rightarrow aBB \text{ к первому } B) \\
&= aaBB \\
&\Rightarrow aa(aBB)B && (\text{применяем } B \rightarrow aBB \text{ к первому } B) \\
&= aaaBBB \\
&\Rightarrow aaa(b)BB && (\text{применяем } B \rightarrow b \text{ к первому } B) \\
&= aaabBB \\
&\Rightarrow aaab(b)B && (\text{применяем } B \rightarrow b \text{ ко второму } B) \\
&= aaabbB \\
&\Rightarrow aaabb(b) && (\text{применяем } B \rightarrow b \text{ к третьему } B) \\
&= aaabbb.
\end{aligned}$$

2 Особенности реализации

2.1 Класс GrammarAnalyzer

Данный класс реализует всю логику LL(1)-анализатора. Он содержит внутренние структуры данных для хранения правил, множеств FIRST и FOLLOW, таблицы разбора, а также методы для их вычисления, синтаксического анализа и генерации цепочек.

Поля класса:

- **productions** — упорядоченное отображение (Map) нетерминалов на списки правых частей правил;
- **startSymbol** — начальный символ грамматики ("S");
- **firstSets** — отображение символа на его множество FIRST;
- **followSets** — отображение нетерминала на его множество FOLLOW;
- **parsingTable** — таблица разбора: для каждого нетерминала и терминала хранится строка правила;
- **semanticAction** — объект, реализующий интерфейс **SemanticActions**, отвечающий за семантические действия.

2.1.1 Метод GrammarAnalyzer (конструктор)

Выполняет начальную инициализацию: создаёт продукции, задаёт семантические действия (генерация машинного кода), затем последовательно вызывает методы `computeFirstSets()`, `computeFollowSets()` и `buildParsingTable()`.

Вход: пустой объект класса.

Выход: объект класса `GrammarAnalyzer` с полностью заполненными структурами.

Работа конструктора начинается с инициализации продукций грамматики (правило для S и два правила для B). Далее создаётся анонимный класс, реализующий интерфейс **SemanticActions**, в котором определён список генерируемых инструкций; метод `execute()` по строке продукции добавляет в список соответствующую команду (PUSH 1, DUP или PUSH 2). После этого вызываются методы вычисления FIRST, FOLLOW и заполнения таблицы разбора.

Листинг 1. Конструктор класса GrammarAnalyzer

```
1 public GrammarAnalyzer() {  
2     productions = new LinkedHashMap<>();  
3     productions.put("S", Arrays.asList(Arrays.asList("a", "B")))  
4     ;  
5     productions.put("B", Arrays.asList(  
6         Arrays.asList("a", "B"),  
7         Arrays.asList("a", "B")  
8     ));  
9 }
```

```

5         Arrays.asList("a", "B", "B"),
6         Arrays.asList("b")
7     ));
8     startSymbol = START;
9     // Семантические действия: генерация кода
10    semanticAction = new SemanticActions() {
11        private final List<String> instructions = new ArrayList
12            <>();
13        @Override
14        public void execute(String production, String
15            actionDescription) {
16            System.out.print(" [Генерация кода] " +
17                actionDescription + " ");
18            switch (production) {
19                case "S → a B": instructions.add("PUSH 1");
20                    System.out.println("PUSH 1"); break;
21                case "B → a B B": instructions.add("DUP");
22                    System.out.println("DUP"); break;
23                case "B → b": instructions.add("PUSH 2");
24                    System.out.println("PUSH 2"); break;
25                default: System.out.println("(нет инструкции)");
26            }
27        }
28        @Override
29        public void reset() { instructions.clear(); }
30        @Override
31        public String getResult() {
32            return "Сгенерированная программа:\n" + String.join(
33                "\n", instructions);
34        }
35    };
36    computeFirstSets();
37    computeFollowSets();
38    buildParsingTable();
39 }

```

2.1.2 Метод computeFirstSets()

Вычисляет множества FIRST для всех символов грамматики итеративным методом до достижения фиксированной точки.

Вход: поля класса.

Выход: заполненное поле `firstSets`.

Сначала для каждого терминала (из массива `TERMINALS`) создаётся множество, содержащее сам терминал (кроме маркера \$). Для нетерминалов инициализируются

пустые множества. Затем в цикле по всем нетерминалам и их правым частям вычисляется FIRST последовательности через вспомогательный метод `firstOfSequence()` и добавляется к текущему FIRST нетерминала. Цикл повторяется, пока происходят изменения.

Листинг 2. Метод `computeFirstSets()`

```
1 private void computeFirstSets() {
2     for (String t : TERMINALS) {
3         Set<String> set = new HashSet<>();
4         if (!t.equals("$")) set.add(t);
5         firstSets.put(t, set);
6     }
7     for (String nt : NONTERMINALS) firstSets.put(nt, new HashSet<>());
8     boolean changed;
9     do {
10        changed = false;
11        for (String nt : NONTERMINALS) {
12            for (List<String> rhs : productions.get(nt)) {
13                Set<String> firstRhs = firstOfSequence(rhs);
14                if (firstSets.get(nt).addAll(firstRhs)) changed
15                    = true;
16            }
17        } while (changed);
18 }
```

2.1.3 Метод `computeFollowSets()`

Вычисляет FOLLOW множества для нетерминалов в соответствии с алгоритмом, описанным в математическом описании.

Вход: поля класса.

Выход: заполненное поле `followSets`.

В начале каждому нетерминалу сопоставляется пустое множество, а начальному символу добавляется маркер конца \$. Затем в цикле просматриваются все правила: для каждого вхождения нетерминала в правой части вычисляется FIRST остатка и в зависимости от наличия в нём добавляются либо термины из FIRST(остатка), либо FOLLOW левой части правила. Цикл повторяется до стабилизации.

Листинг 3. Метод `computeFollowSets()` (фрагмент)

```
1 private void computeFollowSets() {
2     for (String nt : NONTERMINALS) {
3         followSets.put(nt, new HashSet<>());
4     }
5     followSets.get(startSymbol).add("$");
```

```

6         boolean changed;
7     do {
8         changed = false;
9         for (String nt : NONTERMINALS) {
10             for (List<String> rhs : productions.get(nt)) {
11                 for (int i = 0; i < rhs.size(); i++) {
12                     String sym = rhs.get(i);
13                     if (isNonterminal(sym)) {
14                         List<String> beta = rhs.subList(i +
15                             1, rhs.size());
16                         Set<String> firstBeta =
17                             firstOfSequence(beta);
18                         boolean epsilonInFirst = firstBeta.
19                             contains(" ");
20                         for (String term : firstBeta) {
21                             if (!term.equals(" ")) {
22                                 if (followSets.get(sym).add(
23                                     term)) changed = true;
24                             }
25                         }
26                     }
27                     if (epsilonInFirst || beta.isEmpty()
28                         ) {
29                         if (followSets.get(sym).addAll(
30                             followSets.get(nt))) changed
31                             = true; }}}}}
32     } while (changed);
33 }

```

2.1.4 Метод buildParsingTable()

Формирует LL(1)-таблицу разбора на основе вычисленных FIRST и FOLLOW.

Вход: поля класса.

Выход: заполненное поле `parsingTable`.

Для каждого нетерминала и каждой его правой части вычисляется FIRST этой последовательности. Для каждого терминала из FIRST в ячейку таблицы записывается соответствующее правило. Если FIRST содержит `,` то для всех терминалов из FOLLOW нетерминала также записывается это правило. При обнаружении конфликта (два правила на одну ячейку) выводится предупреждение — однако в данной грамматике конфликтов нет.

Листинг 4. Метод buildParsingTable() (фрагмент)

```

1 private void buildParsingTable() {
2     for (String nt : NONTERMINALS) {
3         parsingTable.put(nt, new HashMap<>());
4         for (List<String> rhs : productions.get(nt)) {

```



```

5      Set<String> firstRhs = firstOfSequence(rhs);
6      for (String terminal : firstRhs) {
7          if (!terminal.equals(" ")) {
8              parsingTable.get(nt).put(terminal, nt + " ->
9                  " + String.join(" ", rhs));
10         }
11     }
12     if (firstRhs.contains(" ")) {
13         for (String terminal : followSets.get(nt)) {
14             parsingTable.get(nt).put(terminal, nt + " ->
15                 ");
16         }
17     }
18 }

```

2.1.5 Метод parse(String input)

Запускает детерминированный левый анализ входной строки, выводя на экран состояние стека, текущий символ и выполняемые действия, а также вызывая семантические действия при применении правил.

Вход: строка `input`, состоящая только из символов 'a' и 'b'.

Выход: `true`, если строка принадлежит языку грамматики, иначе `false`.

Метод сбрасывает семантические действия, добавляет к строке маркер конца \$, инициализирует стек символами {\$, S} и устанавливает указатель на первый символ. В цикле, пока стек не пуст, извлекается верхний символ и сравнивается с текущим символом входа. Если они совпадают (оба терминалы), символ снимается со стека и указатель сдвигается. Если верхний символ — нетерминал, то по таблице разбора и текущему символу находится правило; нетерминал заменяется правой частью (в обратном порядке), и выполняется семантическое действие. При ошибке (отсутствие правила или несовпадение терминалов) метод возвращает `false`. При успешном завершении выводится сообщение о принятии строки и сгенерированная программа.

Листинг 5. Метод parse()

```

1 public boolean parse(String input) {
2     semanticAction.reset();
3     List<String> tokens = new ArrayList<>();
4     for (char c : input.toCharArray()) tokens.add(String.
5         valueOf(c));
6     tokens.add("$"); // маркер конца
7     Deque<String> stack = new ArrayDeque<>();
8     stack.push("$");
9     stack.push(startSymbol);
10    int idx = 0;

```

```

10 boolean accept = false;
11 while (!stack.isEmpty()) {
12     String top = stack.peek();
13     String lookahead = tokens.get(idx);
14     System.out.print(stack.toString() + "\t\t" +
15         lookahead + "\t\t");
16     if (top.equals(lookahead)) {
17         stack.pop();
18         idx++; }
19     else if (isNonterminal(top)) {
20         Map<String, String> row = parsingTable.get(top);
21         if (row != null && row.containsKey(lookahead)) {
22             String production = row.get(lookahead);
23             stack.pop();
24             if (!production.endsWith(" ")) {
25                 String rhs = production.split(" → ")[1];
26                 if (!rhs.equals(" ")) {
27                     String[] symbols = rhs.split(" ");
28                     for (int i = symbols.length - 1; i
29                         >= 0; i--) {
30                         stack.push(symbols[i]); } } }
31             String actionDescr = "";
32             if (production.equals("S → a B"))
33                 actionDescr = "Начало вычисления: PUSH 1"
34                 ;
35             else if (production.equals("B → a B B"))
36                 actionDescr = "Операция дублирования верш
37                 ины стека (DUP)";
38             else if (production.equals("B → b"))
39                 actionDescr = "Загрузка константы 2 (PUSH
40                 2)";
41             else actionDescr = " —правило (ничего не ге
42                 нерируем)";
43             semanticAction.execute(production,
44                 actionDescr);
45         } else {return false; } }
46     else {
47         return false; } }
48 accept = (idx == tokens.size());
49 if (accept) {
50     System.out.println("\n Строка ПРИНЯТА.");
51     System.out.println("Семантический результат:\n" +
52         semanticAction.getResult());
53 } else { System.out.println("\n Строка ОТВЕРГНУТА."); }
54 return accept;

```

2.1.6 Метод generate(Random rand)

Генерирует случайную строку, принадлежащую языку грамматики, путём рекурсивного случайного выбора правил.

Вход: объект `Random` для генерации случайных чисел.

Выход: строка из символов 'a' и 'b'.

Метод вызывает вспомогательный рекурсивный метод `generateHelper()` с начальным символом. Если символ — терминал, он возвращается как есть. Если нетерминал, из списка его правил случайно выбирается одно, и для каждого символа правой части рекурсивно вызывается `generateHelper()`. Результаты конкатенируются.

Листинг 6. Метод `generate()` и вспомогательный метод

```
1 public String generate(Random rand) {  
2     return generateHelper(startSymbol, rand);  
3 }  
4 private String generateHelper(String sym, Random rand) {  
5     if (!isNonterminal(sym)) return sym;  
6     List<List<String>> rules = productions.get(sym);  
7     List<String> chosen = rules.get(rand.nextInt(rules.size()));  
8     StringBuilder sb = new StringBuilder();  
9     for (String s : chosen) sb.append(generateHelper(s, rand));  
10    return sb.toString();  
11 }
```

2.2 Класс Main

Содержит точку входа в программу и реализует интерактивное консольное меню, позволяющее пользователю проверять строки, генерировать случайные строки, просматривать FIRST/FOLLOW и таблицу разбора, а также выходить из программы.

2.2.1 Метод main(String[] args)

Организует бесконечный цикл с выводом меню и обработкой команд.

Вход: аргументы командной строки.

Выход: завершение программы по выбору пользователя, вывод в консоль.

Вначале создаются объекты `GrammarAnalyzer`, `Scanner` и `Random`. В цикле выводится меню, и в зависимости от выбора пользователя вызываются соответствующие методы анализатора. При выборе пункта 1 запрашивается строка, проверяется её соответствие алфавиту {a,b}, затем вызывается `parse()`. При выборе 2 генерируется

строка и сразу же разбирается. Пункты 3 и 4 выводят FIRST/FOLLOW и таблицу. Пункт 5 завершает программу.

Листинг 7. Метод main() (фрагмент)

```
1 public static void main(String[] args) {
2     System.out.println("LL(1) анализатор для грамматики: S → a
      B, B → a B B | b");
3     GrammarAnalyzer parser = new GrammarAnalyzer();
4     Scanner scanner = new Scanner(System.in);
5     Random random = new Random();
6     while (true) {
7         System.out.println("\n    Меню:");
8         System.out.println("1. Проверить строку");
9         System.out.println("2. Сгенерировать случайную строку");
10        System.out.println("3. Показать множества FIRST и FOLLOW
      ");
11        System.out.println("4. Показать LL(1) таблицу разбора");
12        System.out.println("5. Выход");
13        System.out.print("Ваш выбор: ");
14        String choice = scanner.nextLine().trim();
15        switch (choice) {
16            case "1": // проверка строки
17                // ...
18                break;
19            case "2": // генерация
20                // ...
21                break;
22            // ...
23        }
24    }
25 }
```

2.3 Интерфейс SemanticActions

Определяет контракт для семантических действий, которые могут быть привязаны к правилам грамматики. Это позволяет отделить логику разбора от конкретных действий (генерация кода, подсчёт, вывод сообщений и т.д.).

Методы интерфейса:

- `void execute(String production, String actionDescription)` — выполняет действие, связанное с продукцией; вызывается при применении правила в процессе разбора.
- `void reset()` — сбрасывает внутреннее состояние (например, очищает список сгенерированных инструкций) перед началом нового разбора.
- `String getResult()` — возвращает результат работы семантических действий после завершения разбора (например, текст программы или итоговые значения).

Листинг 8. Интерфейс SemanticActions

```
1 public interface SemanticActions {  
2     void execute(String production, String actionDescription);  
3     void reset();  
4     String getResult();  
5 }
```

3 Результаты работы

После запуска программы выводится заголовок и меню (Рис. 2).

```
LL(1) анализатор для грамматики: S -> a B, B -> a B B | b

Меню:
1. Проверить строку
2. Сгенерировать случайную строку
3. Показать множества FIRST и FOLLOW
4. Показать LL(1) таблицу разбора
5. Выход
```

Рис. 2. Меню

Предусмотрена защита от некорректного пользовательского ввода (Рис. 3).

```
Меню:
1. Проверить строку
2. Сгенерировать случайную строку
3. Показать множества FIRST и FOLLOW
4. Показать LL(1) таблицу разбора
5. Выход
Ваш выбор: 6
Неверный выбор.

Меню:
1. Проверить строку
2. Сгенерировать случайную строку
3. Показать множества FIRST и FOLLOW
4. Показать LL(1) таблицу разбора
5. Выход
Ваш выбор: ыафы
Неверный выбор.
```

Рис. 3. Некорректный пользовательский ввод

Пункт 2 - генерация корректной строки (Рис. 4).

```
Ваш выбор: 2
Сгенерированная строка: aabb

Разбор строки: "aabb"
Стек    Текущий символ    Действие
[S, $]  a                      Применить: S -> a B
[Генерация кода] Начало вычисления: PUSH 1 -> PUSH 1
[a, B, $] a                      Совпадение a
[B, $]  a                      Применить: B -> a B B
[Генерация кода] Операция дублирования вершины стека (DUP) -> DUP
[a, B, B, $] a                      Совпадение a
[B, B, $] b                      Применить: B -> b
[Генерация кода] Загрузка константы 2 (PUSH 2) -> PUSH 2
[b, B, $] b                      Совпадение b
[B, $]  b                      Применить: B -> b
[Генерация кода] Загрузка константы 2 (PUSH 2) -> PUSH 2
[b, $]  b                      Совпадение b
[$]    $                      Совпадение $

Строка ПРИНЯТА.
Семантический результат:
Сгенерированная программа:
PUSH 1
DUP
PUSH 2
PUSH 2
```

Рис. 4. Пункт меню 2

Пункт 3 - вывод множеств FIRST и FOLLOW (Рис. 5).

```

Ваш выбор: 3

Множества FIRST
FIRST(S) = [a]
FIRST(B) = [a, b]

Множества FOLLOW
FOLLOW(S) = [$]
FOLLOW(B) = [a, b, $]

```

Рис. 5. Пункт меню 3

Пункт 5 - завершение работы программы и вывод сообщения о завершении работы программы (Рис. 6).

```

Ваш выбор: 5
До новых встреч!

```

Рис. 6. Пункт меню 5

Пункт 4 - вывод таблицы разбора (Рис. 7).

```

Ваш выбор: 4
|
LL(1) таблица разбора
НТ      a      b      $
S      S -> a B -      -
B      B -> a B B      B -> b -

```

Рис. 7. Пункт меню 4

Рассмотрим пункт 1 - проверка строки. Введем строку "ab"(Рис. 8).

```

Ваш выбор: 1
Введите строку (только 'a' и 'b'): ab

Разбор строки: "ab"
Стек      Текущий символ  Действие
[S, $]    a              Применить: S -> a B
[Генерация кода] Начало вычисления: PUSH 1 + PUSH 1
[a, B, $] a              Совпадение a
[B, $]    b              Применить: B -> b
[Генерация кода] Загрузка константы 2 (PUSH 2) + PUSH 2
[b, $]    b              Совпадение b
[$]       $              Совпадение $

Строка ПРИНЯТА.
Семантический результат:
Сгенерированная программа:
PUSH 1
PUSH 2
Результат: КОРРЕКТНАЯ строка

```

Рис. 8. Пункт меню 1: строка "ab"

Результат соответствует ожиданиям.

Введем строку "ba"(Рис. 9).

```
Ваш выбор: 1
Введите строку (только 'a' и 'b'): ba

Разбор строки: "ba"
Стек      Текущий символ Действие
[S, $]    b              Ошибка: нет правила для S на символе b
Результат: НЕКОРРЕКТНАЯ строка
```

Рис. 9. Пункт меню 1: строка "ba"

Результат соответствует ожиданиям.

Введем строку "abb"(Рис. 10).

```
Ваш выбор: 1
Введите строку (только 'a' и 'b'): abb

Разбор строки: "abb"
Стек      Текущий символ Действие
[S, $]    a              Применить: S -> a B
[Генерация кода] Начало вычисления: PUSH 1 + PUSH 1
[a, B, $] a              Совпадение a
[B, $]    b              Применить: B -> b
[Генерация кода] Загрузка константы 2 (PUSH 2) -> PUSH 2
[b, $]    b              Совпадение b
[$]       b              Ошибка: неожиданный терминал $
Результат: НЕКОРРЕКТНАЯ строка
```

Рис. 10. Пункт меню 1: строка "abb"

Результат соответствует ожиданиям.

Введем строку "as"(Рис. 11).

```
Ваш выбор: 1
Введите строку (только 'a' и 'b'): as
Недопустимые символы. Используйте только 'a' и 'b'.
```

Рис. 11. Пункт меню 1: строка "as"

Результат соответствует ожиданиям, допустимы только терминалы "a"и "b".

Введем пустую строку - вместо ввода нажмем enter (Рис. 12).

```
Ваш выбор: 1
Введите строку (только 'a' и 'b'):

Разбор строки: ""
Стек      Текущий символ Действие
[S, $]    $              Ошибка: нет правила для S на символе $
Результат: НЕКОРРЕКТНАЯ строка
```

Рис. 12. Пункт меню 1: пустая строка

Результат соответствует ожиданиям.

Введем строку "aaabbb"(Рис. 13).

```

Введите строку (только 'a' и 'b'): aaabbb
Разбор строки: "aaabbb"
Стек      Текущий символ  Действие
[S, $]    a               Применить: S -> a B
[Генерация кода] Начало вычисления: PUSH 1 -> PUSH 1
[a, B, $] a               Совпадение a
[B, $]    a               Применить: B -> a B B
[Генерация кода] Операция дублирования вершины стека (DUP) -> DUP
[a, B, B, $] a            Совпадение a
[B, B, $] a               Применить: B -> a B B
[Генерация кода] Операция дублирования вершины стека (DUP) -> DUP
[a, B, B, B, $] a          Совпадение a
[B, B, B, $] b            Применить: B -> b
[Генерация кода] Загрузка константы 2 (PUSH 2) -> PUSH 2
[b, B, B, $] b            Совпадение b
[B, B, $] b               Применить: B -> b
[Генерация кода] Загрузка константы 2 (PUSH 2) -> PUSH 2
[b, B, $] b               Совпадение b
[B, $]    b               Применить: B -> b
[Генерация кода] Загрузка константы 2 (PUSH 2) -> PUSH 2
[b, $]    b               Совпадение b
[$]       $               Совпадение $

Строка ПРИНЯТА.
Семантический результат:
Сгенерированная программа:
PUSH 1
DUP
DUP
PUSH 2
PUSH 2
PUSH 2
Результат: КОРРЕКТНАЯ строка

```

Рис. 13. Пункт меню 1: строка "aaabbb"

Результат соответствует ожиданиям.

Заключение

В ходе выполнения лабораторной работы для грамматики $S \rightarrow aB, B \rightarrow aBB \mid b$ (которая является s-грамматикой, поскольку все альтернативы каждого нетерминала начинаются с различных терминалов) были построены множества FIRST и FOLLOW для каждого нетерминала, таблица выбора (lookup table), а также матрица отношений предшествования. На их основе разработан детерминированный левый анализатор (LL(1)), так как грамматика является LL(1) и данный метод проще в реализации, чем LR(1). Также реализована генерация цепочек, принадлежащих языку, и назначены семантические действия для заданных продукций (генерация машинно-ориентированного кода, содержащего инструкции PUSH 1, DUP, PUSH 2).

Были выполнены следующие задачи:

1. Построены множества FIRST и FOLLOW для всех нетерминалов грамматики.
2. Построены таблица выбора (lookup table) и матрица отношений предшествования.
3. Реализован детерминированный левый анализатор (стековый LL(1)-парсер) для проверки принадлежности входной цепочки языку.
4. Разработан генератор случайных цепочек, порождаемых грамматикой.
5. Назначены семантические действия для продукций: генерация кода для стековой машины (инструкции PUSH 1, DUP, PUSH 2).
6. Обеспечен вывод подробного лога процесса распознавания (состояние стека, текущий символ, применяемое правило и генерируемая инструкция).

Достоинства реализации: отдельный интерфейс **SemanticActions** позволяет легко заменять или расширять семантику (например, движение робота или вычисление арифметических выражений) без изменения основного алгоритма разбора, вычисление FIRST и FOLLOW выполняется автоматически, без жёстко заданных констант, что позволяет изменять грамматику (при условии, что она остаётся LL(1)).

Недостатки реализации: генератор строк не контролирует длину цепочки, что может привести к очень длинным или очень коротким строкам, отсутствует проверка на то, является ли грамматика LL(1) до построения таблицы.

Возможности масштабирования: добавление новых семантических действий (достаточно реализовать новый класс, имплементирующий интерфейс **SemanticActions**, и передать его в конструктор **GrammarAnalyzer**, например, можно реализовать действия для управления роботом). Также можно модифицировать программу так, чтобы она читала правила из текстового файла и автоматически вычисляла FIRST, FOLLOW и таблицу разбора.

Работа выполнена на языке программирования Java. Всего на работу было потрачено около 17 часов.

Список использованной литературы

- [1] Востров, А.В. Курс лекций по дисциплине «Математическая логика и теория формальных языков». URL <https://tema.spbstu.ru/mathem/> (дата обращения 12.05.2026 г.)
- [2] Про LL-парсинг: Подход к синтаксическому анализу через концепцию нарезания строки. URL <https://habr.com/ru/articles/412905/> (дата обращения 14.05.2026 г.)
- [3] Порождающие грамматики Хомского. URL <https://habr.com/ru/articles/177701/> (дата обращения 15.05.2026 г.)